Python Web Development With Django

Python Django is a web framework that allows to quickly create efficient web pages. Django is also called batteries included framework because it provides built-in features such as Django Admin Interface, default database - SQLite3, etc. When you're building a website, you always need a similar set of components: a way to handle user authentication (signing up, signing in, signing out), a management panel for your website, forms, a way to upload files, etc. Django gives you ready-made components to use.

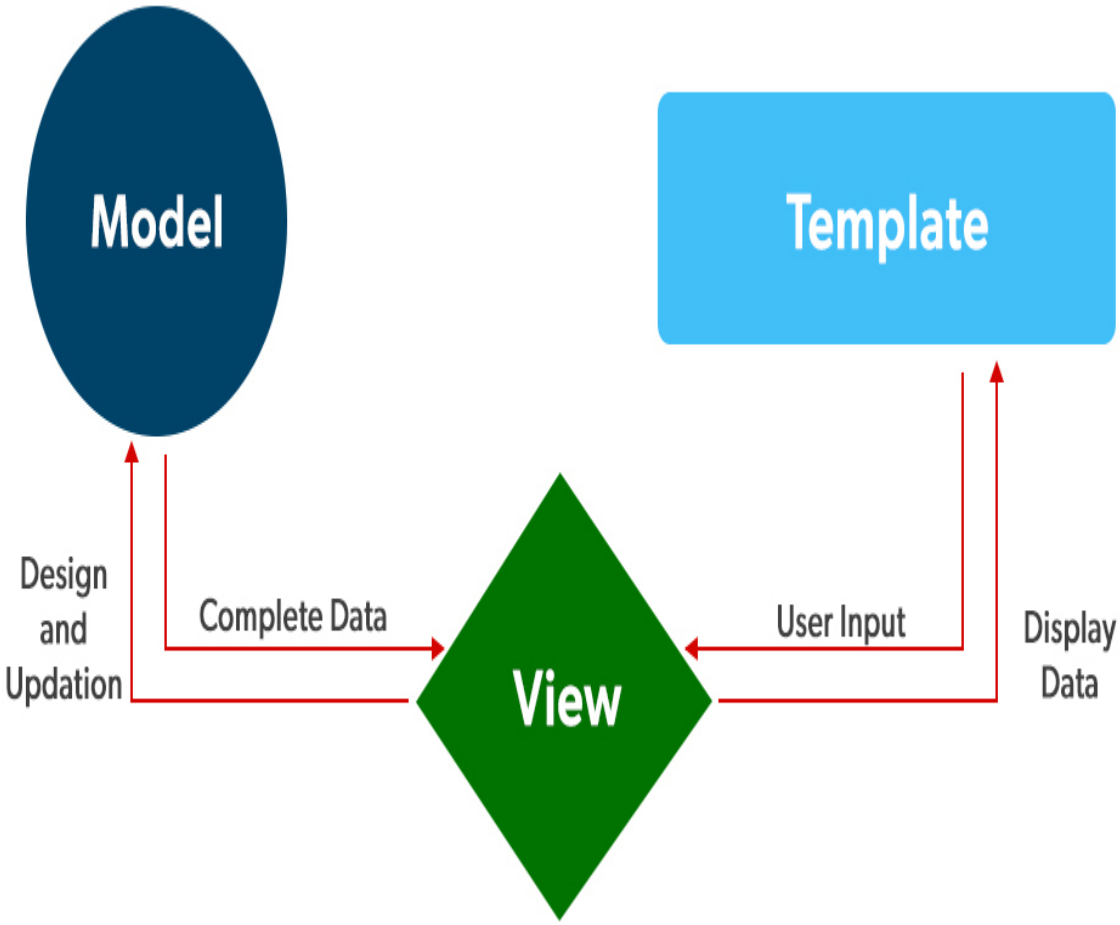
Why Django Framework?

- Excellent documentation and high scalability.
- Used by Top MNCs and Companies, such as Instagram, Disqus, Spotify, Youtube, Bitbucket, Dropbox, etc. and the list is never-ending.
- Easiest Framework to learn, rapid development, and Batteries fully included. Django is a rapid web development framework that can be used to develop fully fleshed web applications in a short period of time.
- The last but not least reason to learn Django is Python, Python has a huge library and features such as Web Scraping, Machine Learning, Image Processing, Scientific Computing, etc. One can integrate all this with web applications and do lots and lots of advanced stuff.

Django Architecture

Django is based on MVT (Model-View-Template) architecture which has the following three parts -

- **Model:** The model is going to act as the interface of your data. It is responsible for maintaining data. It is the logical data structure behind the entire application and is represented by a database (generally relational databases such as MySql, Postgres).
- **View:** The View is the user interface that you see in your browser when you render a website. It is represented by HTML/CSS/Javascript and Jinja files.
- **Template:** A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted. To check more, visit – Django Templates



For more information, refer to Django Project MVT Structure

Setting up the Virtual Environment

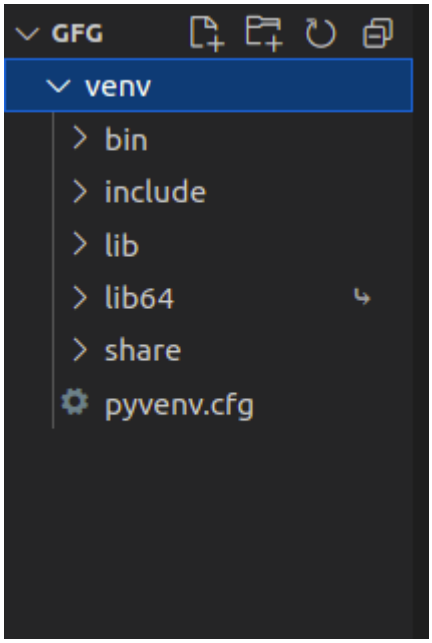
Most of the time when you'll be working on some Django projects, you'll find that each project may need a different version of Django. This problem may arise when you install Django in a global or default environment. To overcome this problem we will use virtual environments in Python. This enables us to create multiple different Django environments on a single computer. To create a virtual environment type the below command in the terminal.

```
python3 -m venv ./name
```

Here the name suggests the name of the virtual environment. Let's create our virtual environment with the name as venv only. So the command to create it will be -

```
python3 -m venv ./venv
```

After running the above command you will see a folder named venv with the following sub-directories.



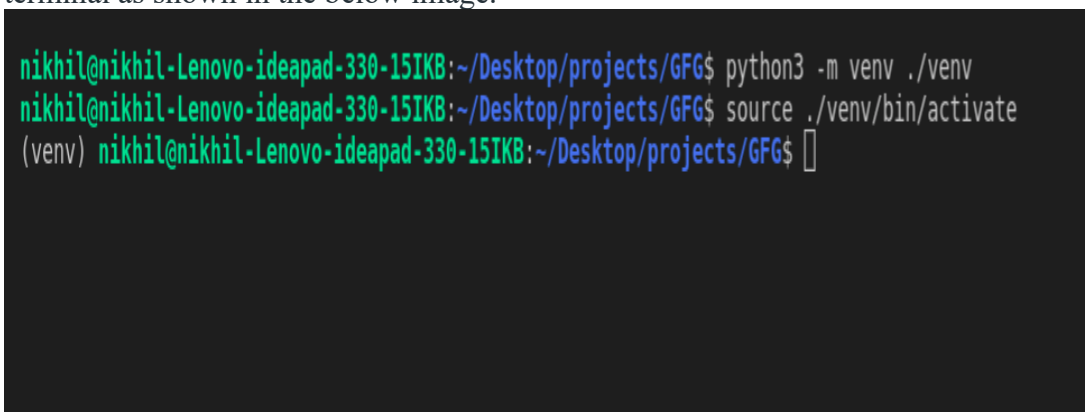
After creating the virtual environment let's activate it. To activate it type the below command in the terminal.

```
source ./venv/bin/activate
```

In the above command **./** is used to tell the current working directory**.**

Note: If you have your virtual environment set up in another location and your terminal opened up in another location, then provide the location to the venv folder i.e. our virtual environment folder.

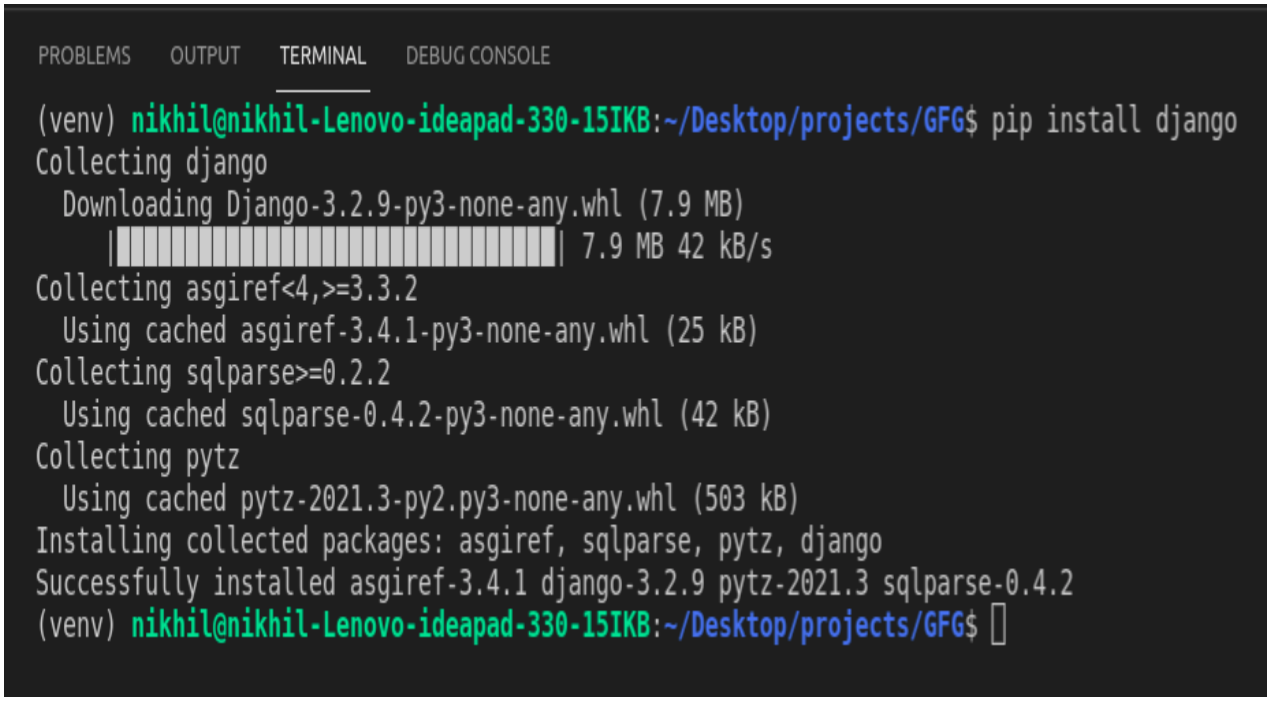
After you run the above command you should see (venv) at the starting of every line of your terminal as shown in the below image.



Installing Django

We can install Django using the pip command. To install this type the below command in the terminal.

```
pip install django
```



For more information, refer to Django Introduction and Installation

Starting the project

- To initiate a project of Django on Your PC, open Terminal and Enter the following command

```
django-admin startproject projectName
```

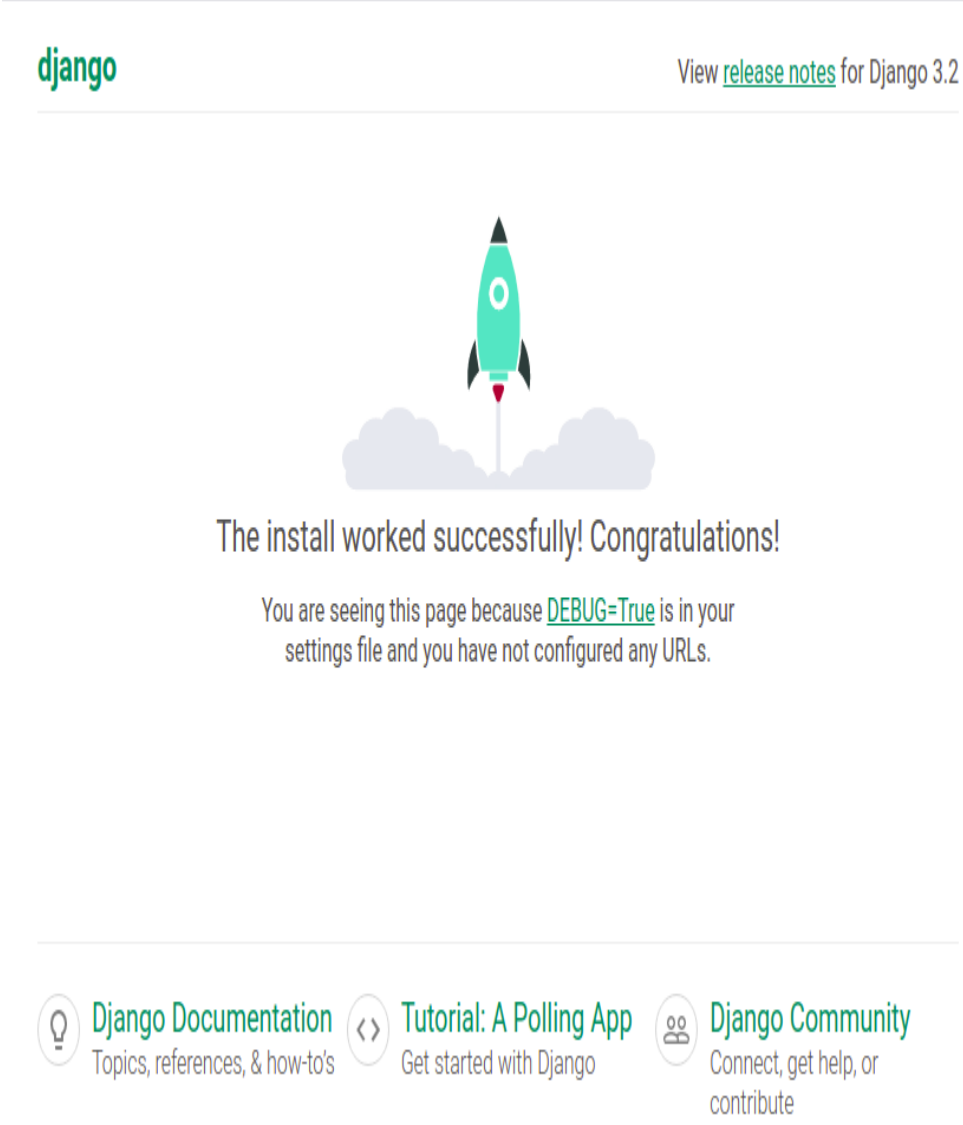
- A New Folder with the name projectName will be created. To enter in the project using the terminal enter command

```
cd projectName
```

- Now let's run the server and see everything is working fine or not. To run the server type the below command in the terminal.

```
python manage.py runserver
```

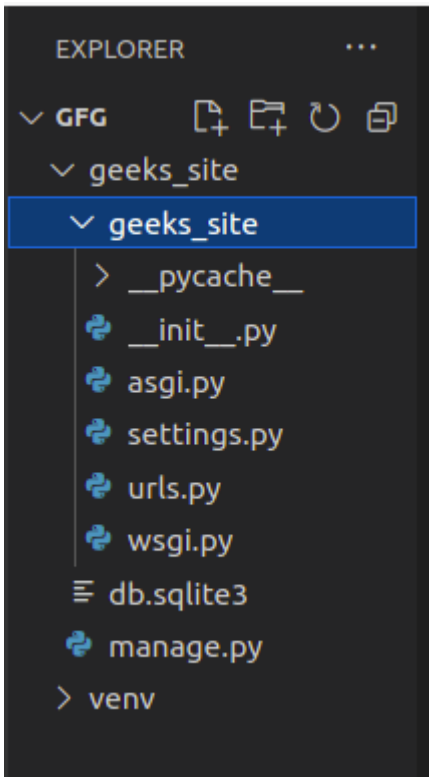
After running the server go to http://127.0.0.1:8000/ and you'll see something like this -



For more information, refer to How to Create a Basic Project using MVT in Django ?

Project Structure

A Django Project when initialized contains basic files by default such as manage.py, view.py, etc. A simple project structure is enough to create a single-page application. Here are the major files and their explanations. Inside the geeks_site folder (project folder) there will be the following files-



Let's discuss these files in detail -
manage.py: This file is used to interact with your project via the command line(start the server, sync the database… etc). For getting the full list of commands that can be executed by manage.py type this code in the command window-

```
python manage.py help
```

- **_init_.py:** It is a python package. It is invoked when the package or a module in the package is imported. We usually use this to execute package initialization code, for example for the initialization of package-level data.
- **settings.py:** As the name indicates it contains all the website settings. In this file, we register any applications we create, the location of our static files, database configuration details, etc.
- **urls.py:** In this file, we store all links of the project and functions to call.
- **wsgi.py:** This file is used in deploying the project in WSGI. It is used to help your Django application communicate with the webserver.

Creating an app
Django is famous for its unique and fully managed app structure. For every functionality, an app can be created like a completely independent module. For example, if you are creating a Blog, Separate modules should be created for Comments, Posts, Login/Logout, etc. In Django, these modules are known as apps. There is a different app for each task. Benefits of using Django apps –

- Django apps are reusable i.e. a Django app can be used with multiple projects.
- We have loosely coupled i.e. almost independent components
- Multiple developers can work on different components
- Debugging and code organization are easy. Django has an excellent debugger tool.
- It has in-built features like admin pages etc, which reduces the effort of building the same from scratch

Django provides some pre-installed apps for users. To see pre-installed apps, navigate to projectName –> projectName –> settings.py. In your settings.py file, you will find

INSTALLED_APPS. Apps listed in INSTALLED_APPS are provided by Django for the developer's comfort.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

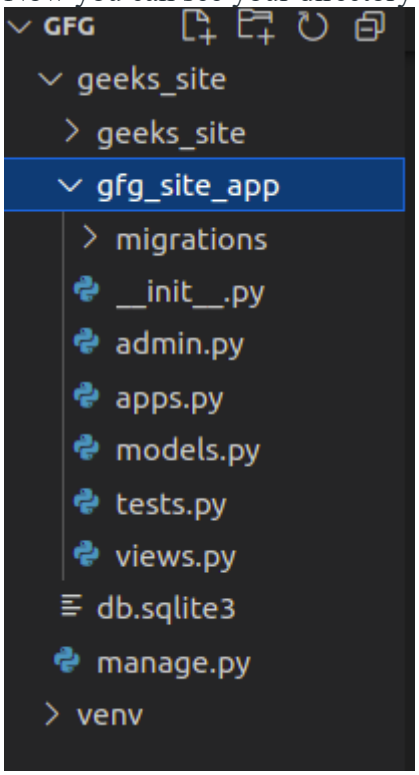
We can also create our own custom apps. To create a basic app in your Django project you need to go to the directory containing manage.py and from there enter the command :

```
python manage.py startapp projectApp
```

Now let's create an app called gfg_site_app, so the command to create the app would be -

```
python manage.py startapp gfg_site_app
```

Now you can see your directory structure as under :



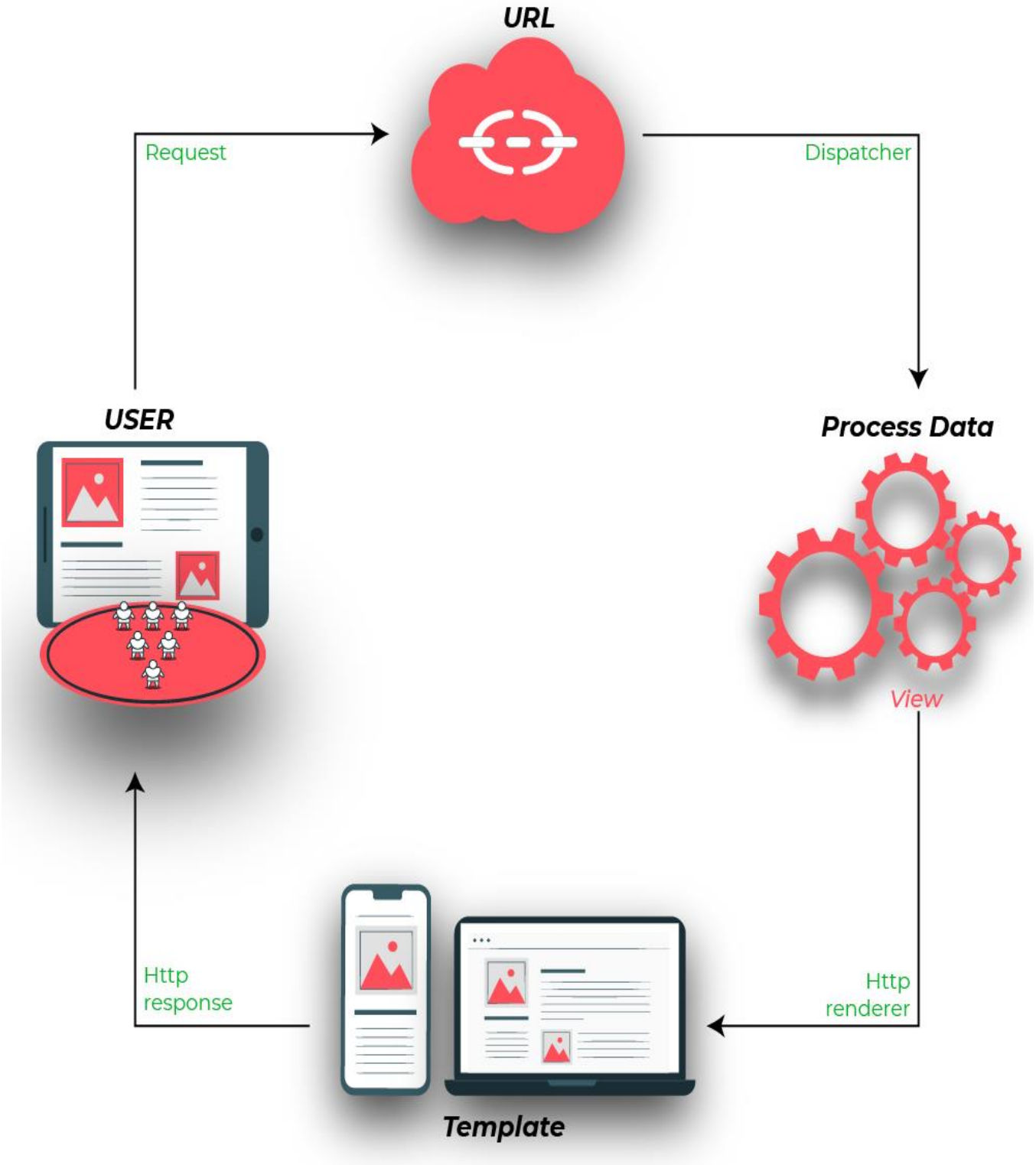
To consider the app in your project you need to specify your project name in the INSTALLED_APPS list as follows in settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'gfg_site_app.apps.GfgSiteAppConfig',
]
```

For more information, refer to How to Create an App in Django ?

Django Views

A view function is a Python function that takes a Web request and returns a Web response. This response can be the HTML contents of a Web page, or a redirect, or a 404 error, or an XML document, or an image, anything that a web browser can display. Django views are part of the user interface — they usually render the HTML/CSS/Javascript in your Template files into what you see in your browser when you render a web page.



Example: Creating View Function

```
from django.http import HttpResponse
```

```
# create a function
def geeks_view(request):

    return HttpResponse("<h1>Welcome to GeeksforGeeks</h1>")
```

Let's step through this code one line at a time:

- First, we import the class HttpResponse from the django.http module, along with Python's datetime library.
- Next, we define a function called geeks_view. This is the view function. Each view function takes an HttpRequest object as its first parameter, which is typically named request.
- The view returns an HttpResponse object that contains the generated response. Each view function is responsible for returning an HttpResponse object.

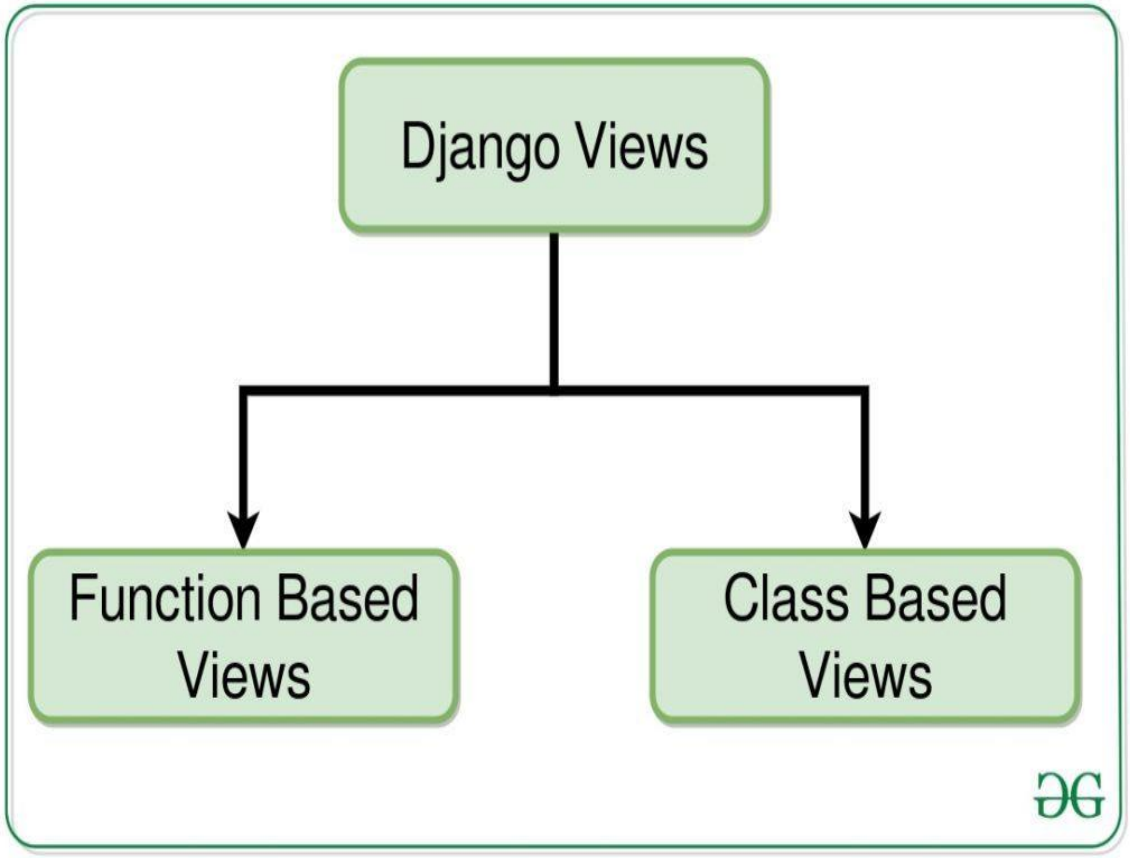
Note: For more info on HttpRequest and HttpResponse visit – Django Request and Response cycle – HttpRequest and HttpResponse Objects

The above Function will render the text Welcome to GeeksforGeeks as h1 on the page. Now the question that may be arising is at what URL this function will be called and how will we handle such URLs. Don't worry we will handle URL in the section but in this section let us continue with the Django views only.

Types of Views

Django views are divided into two major categories:-

- Function-Based Views
- Class-Based Views



Function-Based Views

Function-based views are writer using a function in python which receives as an argument HttpRequest object and returns an HttpResponse Object. Function-based views are generally

divided into 4 basic strategies, i.e., CRUD (Create, Retrieve, Update, Delete). CRUD is the base of any framework one is using for development.
Refer to the below articles to get more information on Function-Based views -

- Django CRUD (Create, Retrieve, Update, Delete) Function Based Views
- Create View
- List View
- Detail View
- Update View
- Delete View

Class-Based Views

Class-based views provide an alternative way to implement views as Python objects instead of functions. They do not replace function-based views, but have certain differences and advantages when compared to function-based views:

- Organization of code related to specific HTTP methods (GET, POST, etc.) can be addressed by separate methods instead of conditional branching.
- Object-oriented techniques such as mixins (multiple inheritances) can be used to factor code into reusable components.

Refer to the below articles to get more information on Class-Based views -

- Class Based Generic Views Django (Create, Retrieve, Update, Delete)
- Createview
- ListView
- DetailView
- UpdateView
- DeleteView
- FormView

Django URL Patterns

In Django, each view needs to be mapped to a corresponding URL pattern. This is done via a Python module called URLConf(URL configuration). Every URLConf module must contain a variable **urlpatterns** which is a set of URL patterns to be matched against the requested URL. These patterns will be checked in sequence until the first match is found. Then the view corresponding to the first match is invoked. If no URL pattern matches, Django invokes an appropriate error handling view.
Now if we see our project we have created an app called gfg_site, the Python module to be used as URLConf is the value of ROOT_URLCONF in gfg_site/settings.py. By default this is set to 'gfg_site.urls'. Every URLConf module must contain a variable urlpatterns which is a set of URL patterns to be matched against the requested URL. These patterns will be checked in sequence, until the first match is found. Then the view corresponding to the first match is invoked. If no URL pattern matches, Django invokes an appropriate error handling view.

URL patterns

Here's a sample code for gfg_site/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.geeks_view, name='geeks_view'),
]
```

Including other URLConf modules

It is a good practice to have a URLConf module for every app in Django. This module needs to be included in the root URLConf module as follows:

```
from django.contrib import admin
```

```
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('gfg_site_app.urls'))
]
```

Now if head towards http://127.0.0.1:8000/ then our site will be -

Welcome to GeeksforGeeks

In the above example, include statement will look into the URLpatterns list in the gfg_site_app/urls.py And then it will look into all the paths defined in the url.py file and will call the respective views function.

Till now we have seen how to show HTML on our website. Now let's suppose we want to use some kind of relational database that, let's say SQLite for our site and we want to create a table in this database and want to link this database to our website. Don't worry we will discuss this in the next section.

Django Models

To tackle the above-said problem Django provides something called Django Models.

A Django model is the built-in feature that Django uses to create tables, their fields, and various constraints. In short, Django Models is the SQL of Database one uses with Django. SQL (Structured Query Language) is complex and involves a lot of different queries for creating, deleting, updating, or any other stuff related to the database. Django models simplify the tasks and organize tables into models. Generally, each model maps to a single database table.

This section revolves around how one can use Django models to store data in the database conveniently. Moreover, we can use the admin panel of Django to create, update, delete or retrieve fields of a model and various similar operations. Django models provide simplicity, consistency, version control, and advanced metadata handling. Basics of a model include –

- Each model is a Python class that subclasses django.db.models.Model.
- Each attribute of the model represents a database field.
- With all of this, Django gives you an automatically-generated database-access API; see Making queries.

Syntax:

```
from django.db import models
class ModelName(models.Model):
      field_name = models.Field(**options)
```

Example:

```
# import the standard Django Model
# from built-in library
from django.db import models
from datetime import datetime
```

```
class GeeksModel(models.Model):

    # Field Names
    title = models.CharField(max_length=200)
    description = models.TextField()
    created_on = models.DateTimeField(default=datetime.now)
    image = models.ImageField(upload_to="images/%Y/%m/%d")

    # rename the instances of the model
    # with their title name
    def __str__(self) -> str:
        return self.title
```

Whenever we create a Model, Delete a Model, or update anything in any of models.py of our project. We need to run two commands makemigrations and migrate. makemigrations basically generates the SQL commands for preinstalled apps (which can be viewed in installed apps in settings.py) and your newly created app's model which you add in installed apps whereas migrate executes those SQL commands in the database file.

So when we run,

```
Python manage.py makemigrations
```

SQL Query to create above Model as a Table is created and

```
Python manage.py migrate
```

creates the table in the database.

Now we have created a model we can perform various operations such as creating a Row for the table or in terms of Django Creating an instance of Model. To know more visit – Django Basic App Model – Makemigrations and Migrate.

Now let's see how to add data to our newly created SQLite table.

Django CRUD – Inserting, Updating, and Deleting Data

Django lets us interact with its database models, i.e. add, delete, modify and query objects, using a database-abstraction API called ORM(Object Relational Mapper). We can access the Django ORM by running the following command inside our project directory.

```
python manage.py shell
```

Adding objects

To create an object of model Album and save it into the database, we need to write the following command:

```
from gfg_site_app.models import GeeksModel

obj = GeeksModel(title="GeeksforGeeks",
   description="GFG is a portal for computer science students")
obj.save()
```

Retrieving objects

To retrieve all the objects of a model, we write the following command:

```
GeeksModel.objects.all()
```

Output:

```
<QuerySet [<GeeksModel: GeeksforGeeks>]>
```

Modifying existing objects

We can modify an existing object as follows:

```
obj = GeeksModel.objects.get(id=1)
obj.title = "GFG"
obj.save()
```

```
GeeksModel.objects.all()
```
Output:
```
<QuerySet [<GeeksModel: GFG>]>
```
Deleting objects

To delete a single object, we need to write the following commands:
```
obj = GeeksModel.objects.get(id=1)
obj.delete()

GeeksModel.objects.all()
```
Output:
```
(1, {'gfg_site_app.GeeksModel': 1})
<QuerySet []>
```
Refer to the below articles to get more information about Django Models -

- Django Models
- ORM – Inserting, Updating & Deleting Data
- Basic App Model – Makemigrations and Migrate
- model data types and fields list
- Add the slug field inside Django Model
- Intermediate fields in Django
- Uploading images in Django
- Change Object Display Name using __str__ function – Django Models
- Built-in Field Validations – Django Models
- Custom Field Validations in Django Models
- How to use Django Field Choices ?
- Overriding the save method – Django Models

Uploading Images in Django

When defining the models we used the ImageField for uploading images and we wrote the upload_to parameter as upload_to="images/%Y/%m/%d") because this will create a directory data structure of the format image>>year>>month>>date so that tracking images may become easier.

Before uploading the image we need to write the below code in the setting.py file.
```
MEDIA_ROOT =  BASE_DIR/'media'
MEDIA_URL = '/media/'
```
- MEDIA_ROOT is for server path to store files in the computer.
- MEDIA_URL is the reference URL for browser to access the files over Http

In the urls.py we should edit the configuration like this
```
from django.conf import settings
from django.conf.urls.static import static
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)
```
Let's see how to upload data into the model using Django Admin Interface.

Render a model in Django Admin Interface

To render a model in Django admin, we need to modify app/admin.py. Go to admin.py in geeks_site_app and enter the following code. Import the corresponding model from models.py and register it to the admin interface.
```
from django.contrib import admin
from .models import GeeksModel

# Register your models here.
```

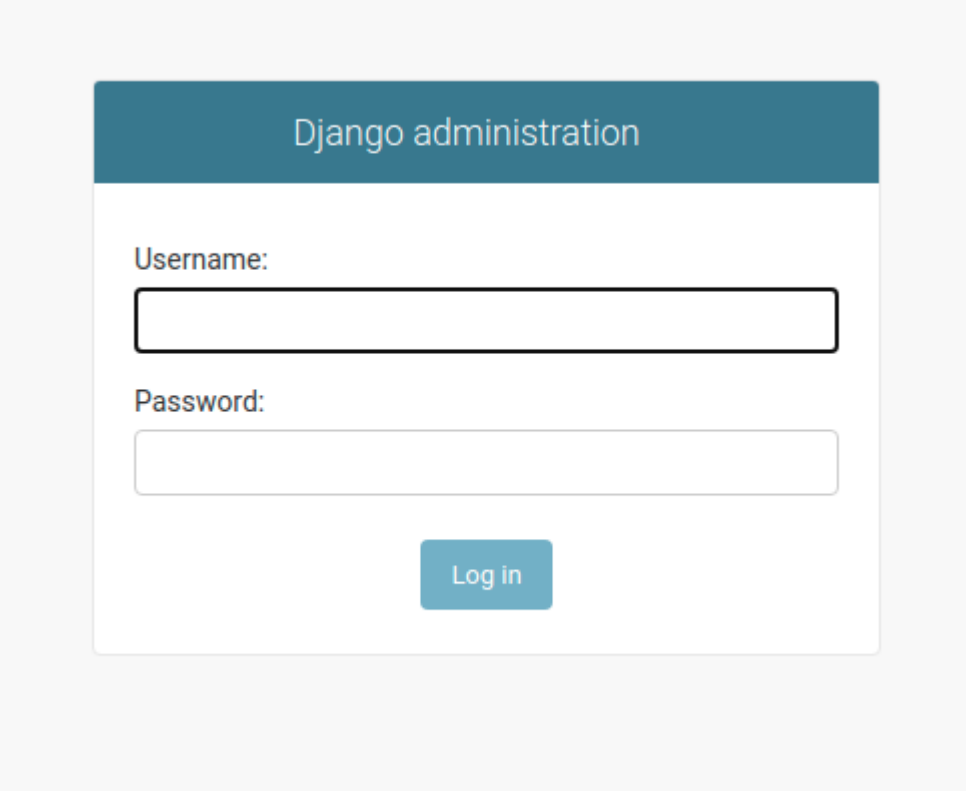
```
admin.site.register(GeeksModel,)
```

Now let's create a superuser for our project that can have access to the admin area of our site. To create a super user type the below command -

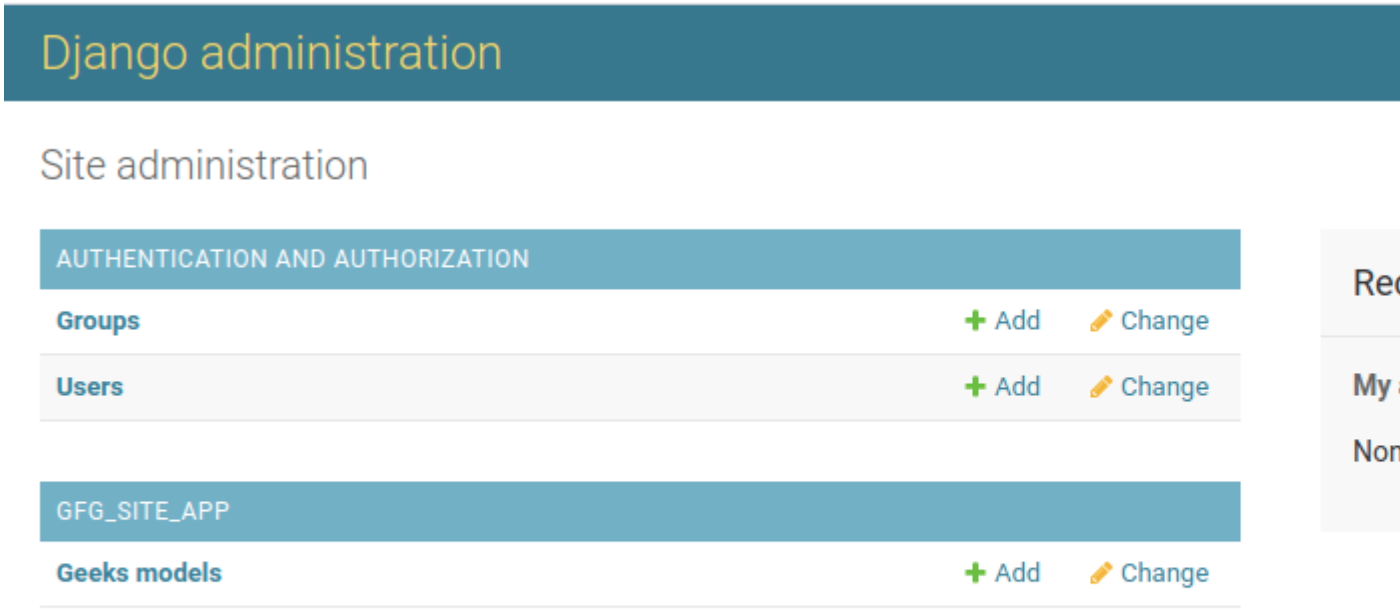
```
python manage.py createsuperuser
```

```
(venv) nikhil@nikhil-Lenovo-ideapad-330-15IKB:~/Desktop/projects/GFG/geeks_site$ python manage.py createsuperuser
Username (leave blank to use 'nikhil'):
Email address: nikhil@gmail.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(venv) nikhil@nikhil-Lenovo-ideapad-330-15IKB:~/Desktop/projects/GFG/geeks_site$
```

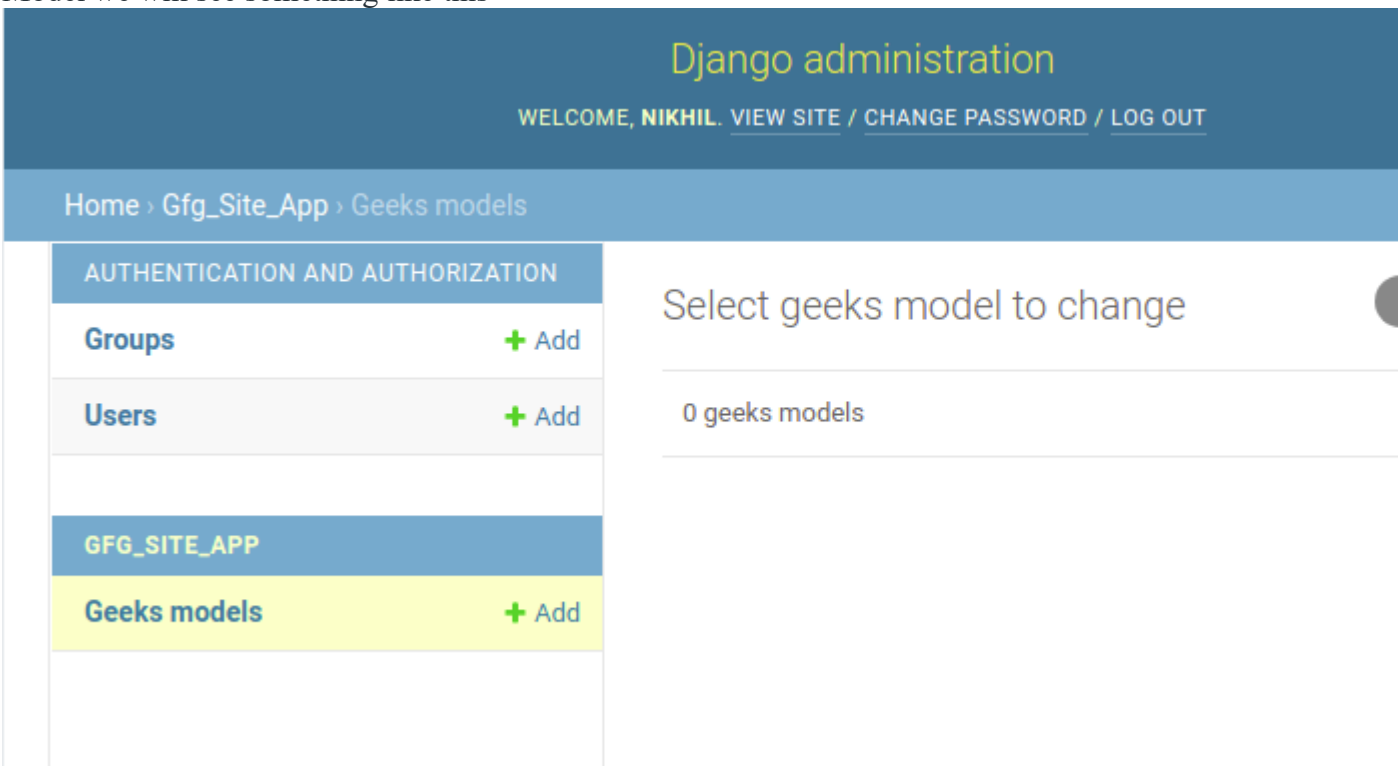
Now go to http://127.0.0.1:8000/admin on the browser to access the admin interface panel.



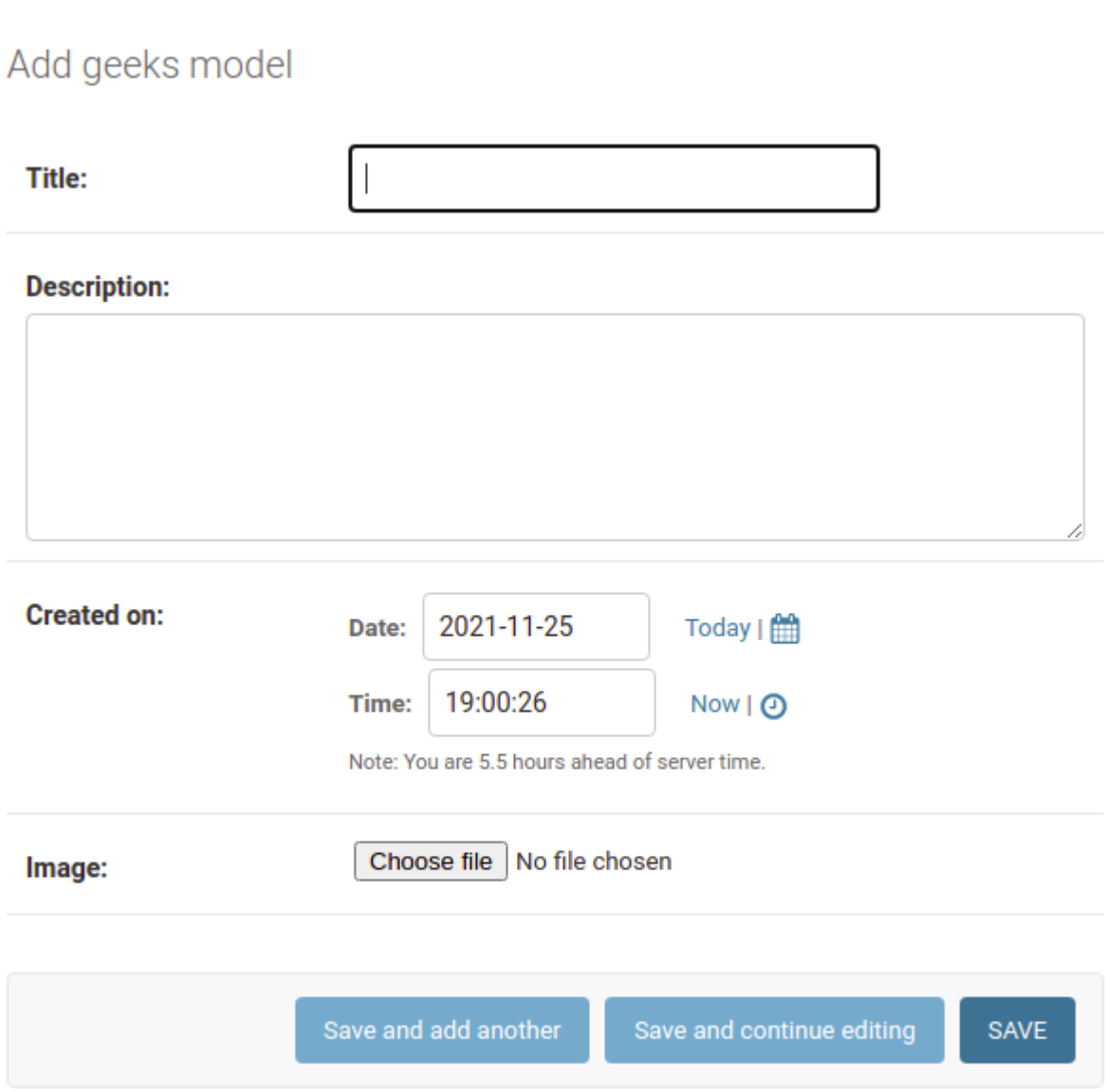
Give the username and password created for superuser and then the admin dashboard will open and there we will be able to see our Geeks models that we just created.



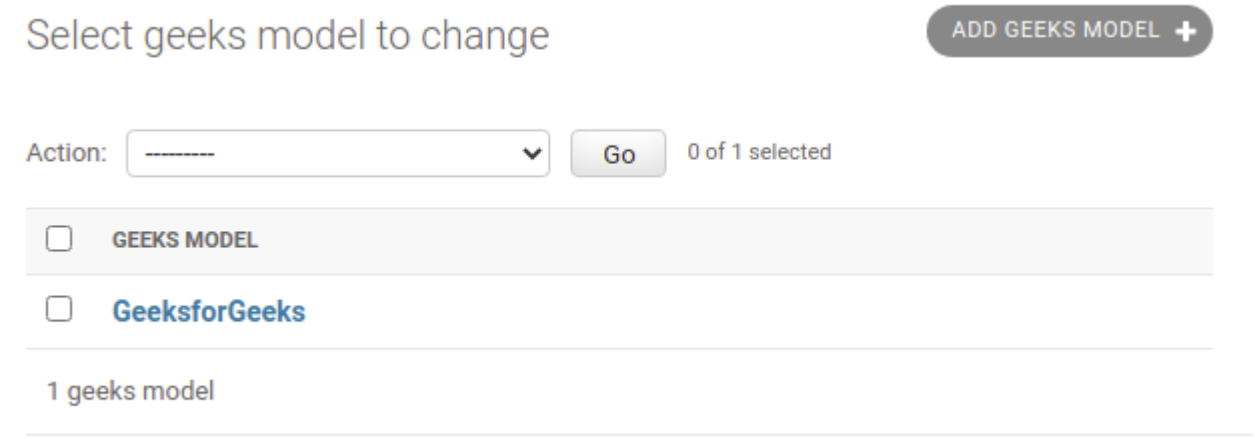
Note: For more information refer to Render Model in Django Admin Interface.
Now let's see how to enter data using the admin dashboard. Now clicking on the Geeks Model we will see something like this -



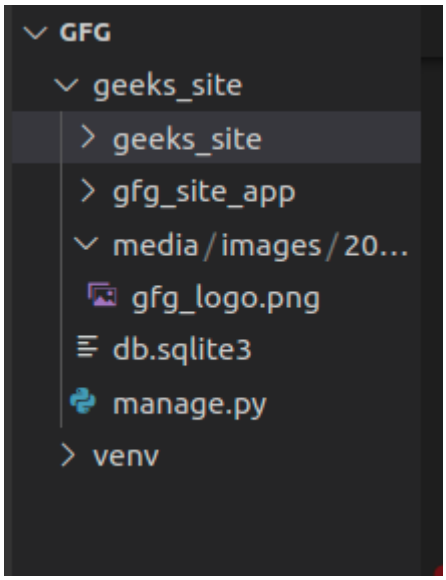
We can click on the Add Geeks Model button on the right top corner and then we will be able to see the fields for adding data. See the below image -



After adding the required data and the image field we will see something like this on our admin dashboard -



You can also see the media folder in your code editor -



Connecting Django to different Database

Django comes built-in with the SQLite database. We can also see this in the DATABASES dictionary in our settings.py file.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

If you want to change this to another database you can change the above dictionary. Let's suppose we want to change this database to PostgreSQL. Assuming the required dependencies are installed and the PostgreSQL is set up then the DATABASES dictionary will look like -

```
DATABASES = {
  'default': {
      'ENGINE': 'django.db.backends.postgresql',
      'NAME': '<database_name>',
      'USER': '<database_username>',
```

```
        'PASSWORD': '<password>',
        'HOST': '<database_hostname_or_ip>',
        'PORT': '<database_port>',
    }
}
```

Refer to the below articles to get more information about connecting Django to different databases -

- How to use PostgreSQL Database in Django?
- Connect Django Project to MongoDB using Django

Django Templates

Templates are the third and most important part of Django's MVT Structure. A template in Django is basically written in HTML, CSS, and Javascript in a .html file. Django framework efficiently handles and generates dynamically HTML web pages that are visible to the end-user. Django mainly functions with a backend so, in order to provide a frontend and provide a layout to our website, we use templates. There are two methods of adding the template to our website depending on our needs.

- We can use a single template directory which will be spread over the entire project.
- For each app of our project, we can create a different template directory.

For our current project, we will create a single template directory that will be spread over the entire project for simplicity. App-level templates are generally used in big projects or in case we want to provide a different layout to each component of our webpage.

Configuration

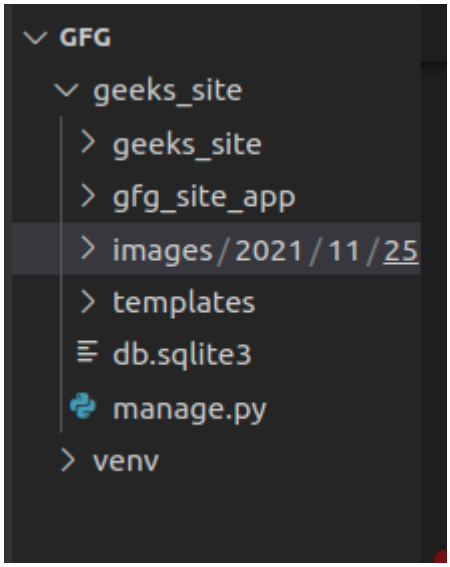
Django Templates can be configured in app_name/settings.py,

```
TEMPLATES = [
    {
        # Template backend to be used, For example Jinja
        'BACKEND': 'django.template.backends.django.DjangoTemplates',

        # directories for templates
        'DIRS': [],
        'APP_DIRS': True,

        # options to configure
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Now let's create a template directory and add that directory in the above configuration. After creating the templates folder our directory should look like this -



Let's add the location of this directory in our templates dictionary.

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',

        # adding the location of our templates directory
        'DIRS': [BASE_DIR/"templates"],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

After adding the location of the template directory we will create a simple HTML file and name it as index.html and then we will render this file from our view function.

HTML file:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Homepage</title>
</head>
<body>
    <h1>Welcome to Geeksforgeeks</h1>
</body>
</html>
```

To render this HTML on our site we need to use the render function from the django.shortcuts. Below is the updated view function.

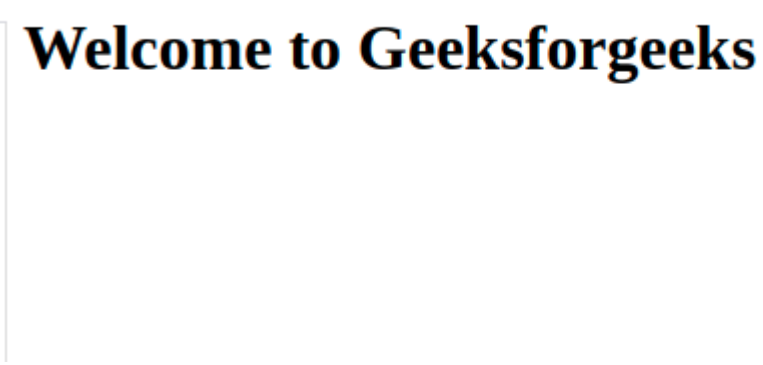
views.py

```
from django.shortcuts import render

# create a function
def geeks_view(request):

    return render(request, "index.html")
```

If we head to our website we will see the HTML data on our site as -



The Django Templates not only show static data but also the data from different databases connected to the application through a context dictionary. Let's see this with an example. We will try to render the content of our database dynamically to our website.

First, let's update our views.py file. In this file we will get our data from our database and then pass this database as a dictionary to our HTML file.

views.py

```
from django.shortcuts import render
from .models import SampleModel

# create a function
def geeks_view(request):

    content = SampleModel.objects.all()
    context = {
        'content': content
    }
    return render(request, "index.html", context=context)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Homepage</title>
</head>
<body>

    {% for data in content %}
```

```
    <h2>{{data.title}}</h2>
    <img src="{{ data.image.url }}" alt="">

<p><strong>Description:</strong>{{data.description}}</p>


<p><strong>Created On:</strong>{{data.created_on}}</p>

    {% endfor %}

</body>
</html>
```

Our website now looks like this -

Now if we add more data to our site then that data will also be shown to our site without making any changes to our HTML or views.py. Let's add some data and then see if it works or not.

Django template language

This is one of the most important facilities provided by Django Templates. A Django template is a text document or a Python string marked-up using the Django template language. Some constructs are recognized and interpreted by the template engine. The main ones are variables and tags. As we used for the loop in the above example, we used it as a tag. similarly, we can use various other conditions such as if, else, if-else, empty, etc. The main characteristics of Django Template language are Variables, Tags, Filters, and Comments.

Variables

Variables output a value from the context, which is a dict-like object mapping keys to values. The context object we sent from the view can be accessed in the template using variables of Django Template.

Syntax

```
{{ variable_name }}
```

Tags

Tags provide arbitrary logic in the rendering process. For example, a tag can output content, serve as a control structure e.g. an “if” statement or a “for” loop, grab content from a database, or even enable access to other template tags.

Syntax

```
{% tag_name %}
```

Filters

Django Template Engine provides filters that are used to transform the values of variables and tag arguments. We have already discussed major Django Template Tags. Tags can’t modify the value of a variable whereas filters can be used for incrementing the value of a variable or modifying it to one’s own need.

Syntax

```
{{ variable_name | filter_name }}
```

Comments

Template ignores everything between {% comment %} and {% end comment %}. An optional note may be inserted in the first tag. For example, this is useful when commenting out code for documenting why the code was disabled.

Syntax

```
{% comment 'comment_name' %}
{% endcomment %}
```

Template Inheritance

The most powerful and thus the most complex part of Django's template engine is template inheritance. Template inheritance allows you to build a base "skeleton" template that contains all the common elements of your site and defines blocks that child templates can override. extends tag is used for the inheritance of templates in Django. One needs to repeat the same code again and again. Using extends we can inherit templates as well as variables.

Syntax

```
{% extends 'template_name.html' %}
```

Example: Assume the following directory structure:

```
dir1/
   template.html
   base2.html
   my/
      base3.html
base1.html
```

In template.html, the following paths would be valid:

```
{% extends "./base2.html" %}
{% extends "../base1.html" %}
{% extends "./my/base3.html" %}
```

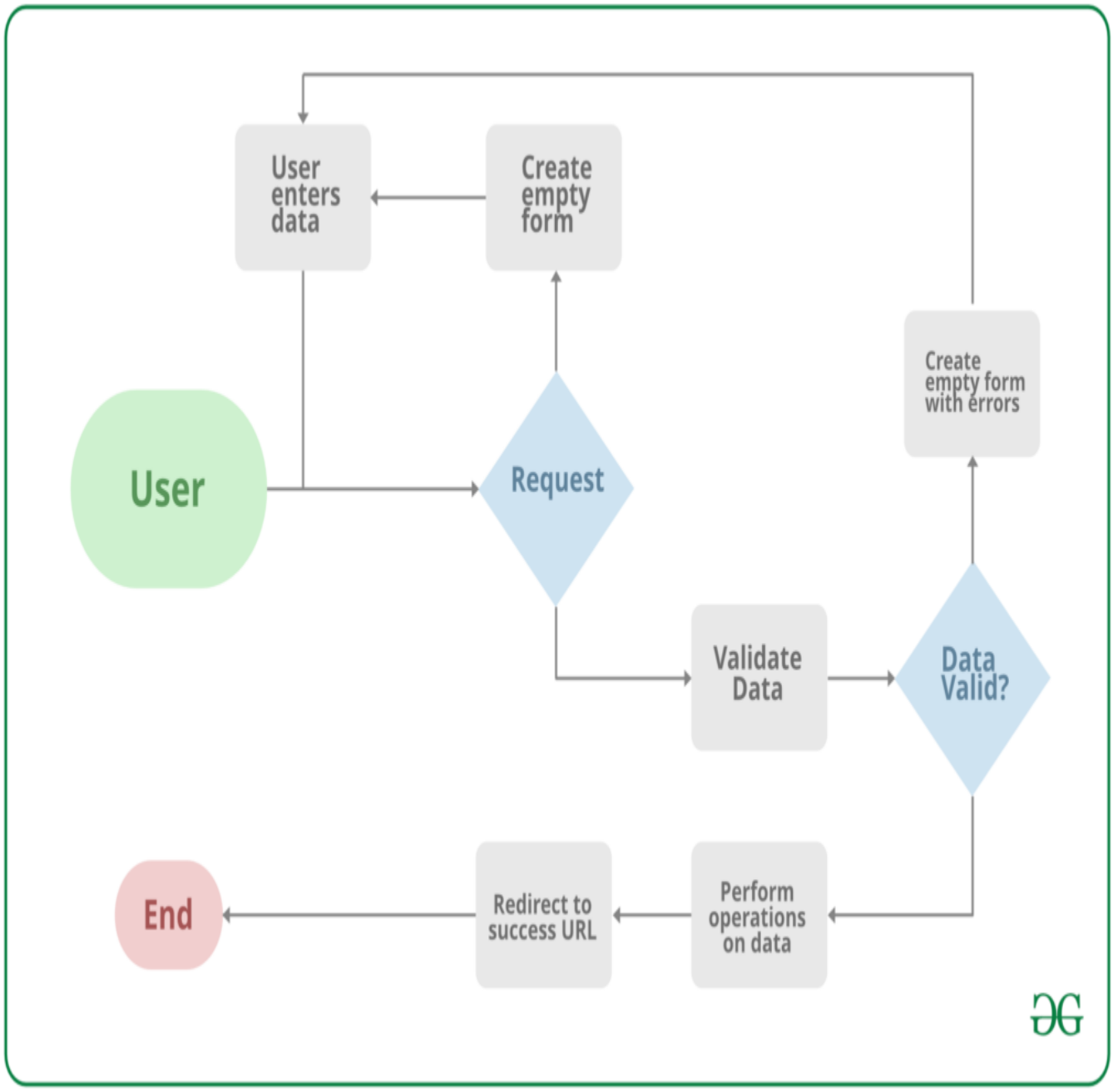
Refer to the below articles to get more information about Django Templates -

- Boolean Operators
- for loop
- if – Django Templates
- Template Inheritance

Django Forms

When one creates a Form class, the most important part is defining the fields of the form. Each field has custom validation logic, along with a few other hooks. Forms are basically used for taking input from the user in some manner and using that information for logical operations on databases. For example, Registering a user by taking input as his name, email, password, etc. Django maps the fields defined in Django forms into HTML input fields. Django handles three distinct parts of the work involved in forms:

- preparing and restructuring data to make it ready for rendering
- creating HTML forms for the data
- receiving and processing submitted forms and data from the client



Note: All types of work done by Django forms can be done with advanced HTML stuff, but Django makes it easier and efficient especially the validation part. Once you get hold of Django forms you will just forget about HTML forms.

Creating Django Forms

Creating a form in Django is completely similar to creating a model, one needs to specify what fields would exist in the form and of what type. For example, to input, a registration form one might need First Name (CharField), Roll Number (IntegerField), and so on.

To create a Django form, first create a forms.py inside the app folder.

```
from django import forms

class GeeksForm(forms.Form):
```

```
    title = forms.CharField(max_length=200)
    description = forms.CharField(widget=forms.Textarea)
    image = forms.ImageField()
```

Let's create a different view function for handling forms and we will map this view function to a different URL. In the above created views.py file import the GeeksForm from the forms.py and create the below function.

views.py

```
from .forms import GeeksForm

def geeks_form(request):
    context = {}
    context['form'] = GeeksForm
    return render(request, "form.html", context=context)
```

Map this function to a different URL let's say we will map this function to the http://127.0.0.1:8000/add/. To do this go to urls.py file of the app and another path for above URL.

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.geeks_view, name='geeks_view'),
    path('add/', views.geeks_form, name="geeks_form")
]
```

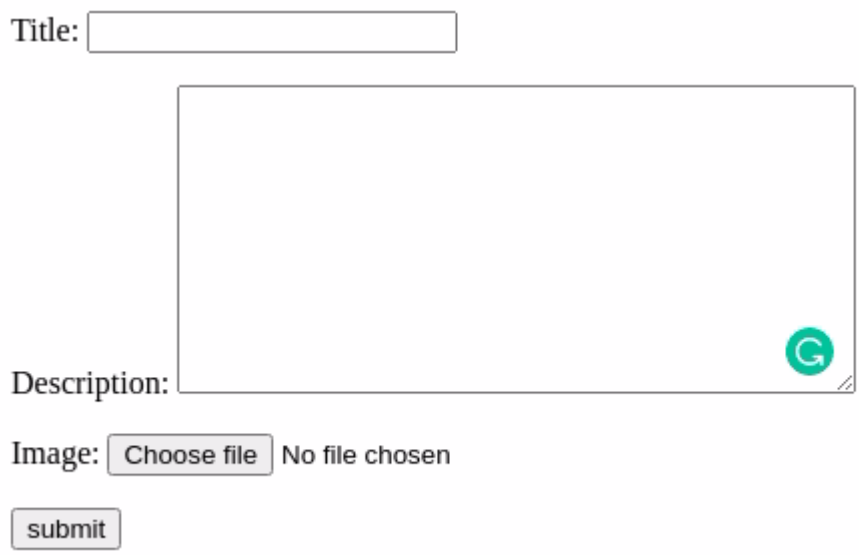
Django form fields have several built-in methods to ease the work of the developer but sometimes one needs to implement things manually for customizing User Interface(UI). A form comes with 3 in-built methods that can be used to render Django form fields.

- {{ form.as_table }} will render them as table cells wrapped in <tr> tags
- {{ form.as_p }} will render them wrapped in <p> tags
- {{ form.as_ul }} will render them wrapped in <li> tags

Now let's make the form.html for rendering our form.

```
<form action="" method="POST">
    {% csrf_token %}
    {{form.as_p}}
    <input type="submit" value="submit">
</form>
```

After doing this save all the files and go to http://127.0.0.1:8000/add/ to see the form we created. It should look like this -



We can also see that our form is validated automatically. We cannot submit an empty form.

Create Django Form from Models

Django ModelForm is a class that is used to directly convert a model into a Django form. To create a form directly for our model, dive into forms.py and Enter the following -

```
from django import forms
from .models import GeeksModel

class GeeksForm(forms.ModelForm):

    class Meta:
        model = GeeksModel
        fields = ['title', 'description', 'image']
```

Now visit http://127.0.0.1:8000/add/ you will see the same form as above but with less code. Both the Django forms we created are similar but the only difference is the save() method. Every ModelForm has a save() method which saves the database object from the data bound to the form. In simpler words we will be able to save the data to our database using the ModelForm only. For this change the view method as follow -

views.py

```
def geeks_form(request):
    if request.method == 'POST':
        form = GeeksForm(request.POST, request.FILES)

        if form.is_valid():
            form.save()
            return redirect("geeks_view")
        else:

            # uncomment the below line to see errors
            # in the form (if any)
            # print(form.errors)
            return redirect("geeks_form")
    else:
        context = {}
```

```
    context['form'] = GeeksForm
    return render(request, "form.html", context=context)
```